Software Engineering Project - Simulated E-Banking System (SEBS)

CSC445 - Software Engineering

Professor Gao

Team 3 - Dylan Chamberlin, Regan Cunningham, Jacob Dell, and Aiden Grimsey

Project Overview

Details:

- We created a web-based simulated online banking system
- It allows for users to manage accounts and transactions with their banking details
- It includes both customer and administrator roles
- It is designed ONLY for educational and demonstration purposes, it does not use real financial data or an actual external banking system

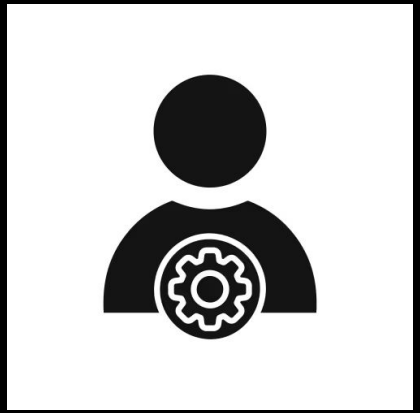
Objectives:

- Apply the Software Development Life Cycle (SDLC)
- Translate all the requirements into a working system with a frontend and a backend
- Design a model that was secure and practical
- Practice teamwork and collaboration
    - Through communication and Github, with version control and access management
- Communicated technical design and implementation clearly

System Scope

We want the **administrator** to be able to:

- Monitor the system activity
- Be able to view and manage customer user accounts



We want the **customer** to be able to:

- Log in securely, with their information stored and protected for later logins
- View their account balances
- Transfer funds between numerous accounts
- View their transaction history on any account they have

Key Features

1

User authentication system

Users are able to log in and log out at their discretion, their information will be stored securely

2

Account dashboard

Users can view their transaction history, view their account balances, and transfer/deposit/with draw money

3

Transaction operations

- Deposit
- Withdrawal
- Transfer

4

Transaction history tracking

5

Administrative Monitoring tools

User Roles

Access Control: Role-based permissions that restrict functionality

Customer

- Access personal accounts
- Perform transactions
- View account activity

Administrator

- View system logs
- Manage accounts
- Monitor transactions made between accounts

Functional Requirements

Customer

Authentication
- Login with username and password
- Validate credentials and grant/deny access accordingly
- Show an error message on failed login

Account Summary
- Dashboard showing all accounts with current balance, account name, and number

Transaction History
- View transaction history per account
- Each record shows date, type, and amount

Transfers
- Select source and destination accounts
- Enter an amount, verify sufficient funds
- Reject and show error if funds are insufficient
- Update both account balanced and record the transfer in both transaction histories

Deposits and Withdrawals
- Perform simulated deposits and withdrawals on a selected account
- Require an amount, check funds before withdrawal
- Update balance and record in the transaction history

Administrator

- Create and modify simulated customer accounts
- View system activity, transaction records, and transaction logs
- All admin functions restricted to administrator-role users only

Non-Functional Requirements

Reliability

- Account balances and transaction records must stay consistent during normal operation
- Failed or incomplete transactions must not modify any stored data

Availability

- System is available whenever the local server is running
- The system in accessible through a web browser when active

Security

- Require authentication before accessing any account functions
- Restrict admin functions to admin-role users
- Store credentials securely

Performance

- Respond to requests within a reasonable time under normal conditions
- Update balances and records immediately after a completed operation

Maintainability

- Source code in version control
- Documentation kept up to date

Design Constraints

- Locally hosted only, there is NO connection to any real banking system or financial network
- All the data is simulated

System Architecture

Presentation Layer

HTML interface

- User interaction

Application Layer

Django backend

- Business logic

Data Layer

SQLite database

- Data storage

Technology Stack

Frontend:
HTML

Backend:
Python (Django framework)

Database:
SQLite

Version Control:
Github

Database Design/Tables

UserProfile

- Already extends Django's built in User with a banking role
  - **user** - link to Django's auth user
  - **role** - either "customer" or "administrator"

Account

- Represents a bank account owned by a user
  - **owner** - linked to a user
  - **account_number** - unique string, max 20 chars
  - **account_name** - display name
  - **account_type** - either "checking" or "savings"
  - **balance** - decimal, up to 12 digits
  - **created_at** - auto-set timestamp when created

Transaction

- A record of every financial event on an account
  - **account** - the account this transaction belongs to
  - **transaction_type** - one of **deposit**, **withdrawal**, **transfer_in**, **transfer_out**
  - **amount** - decimal value
  - **transaction_date** - auto-set timestamp
  - **source_account** - optional set on withdrawals and transfers
  - **destination_account** - optional, set on deposits and transfers
  - **description** - short text description of the transaction

ActivityLog

- An audit trail of user actions across the whole system
  - **user** - which user did the action
  - **action_type** - one of **login**, **failed_login**, **logout**, **deposit**, **withdrawal**, **transfer**, **account_created**, **account_updated**
  - **action_datetime** - auto-set timestamp
  - **details** - free text with any specifics

Database Functions

deposit_to_account(user, account_id, amount):

Adds money to an account. It verifies the account belongs to the user, adds the amount to the balance, created a **Transaction** record of type **deposit**, and logs it to **ActivityLog**

withdraw_from_account(user, account_id, amount):

Removes money from an account. Same ownership check, then it verifies there are sufficient funds before subtracting. Creates a **withdrawal** transaction record and logs it

transfer_between_accounts(user, source_account_id, destination_account_id, amount)
Moves money between two of the user's accounts. Checks that source and destination are different, verifies sufficient funds, then updates boh balances in one atomic operation. Creates two transaction records, one **transfer_out** and one **transfer_in** on the destination, and logs it

Database Functions Continued

Shared Behavior

- All wrapped in **@transaction.atomic** so if anything fails, the whole operation starts over, so no transaction is half-completed.
- All database functions use **select_for_update()** when fetching accounts to stop race conditions if two requests happen at the same time
- All three run the amount being transferred through _clean_amount() first, which will reject anything with a non-numeric input and anything less than 0.
- All three raise the exception **BankingError** on failure, which the views will catch and display as an error message to the user.

Data Design

User:

Login credentials and role

Account:

Account number, type, balance

Transaction:

Deposit, withdrawal, transfer records

Activity Log

Tracks system events

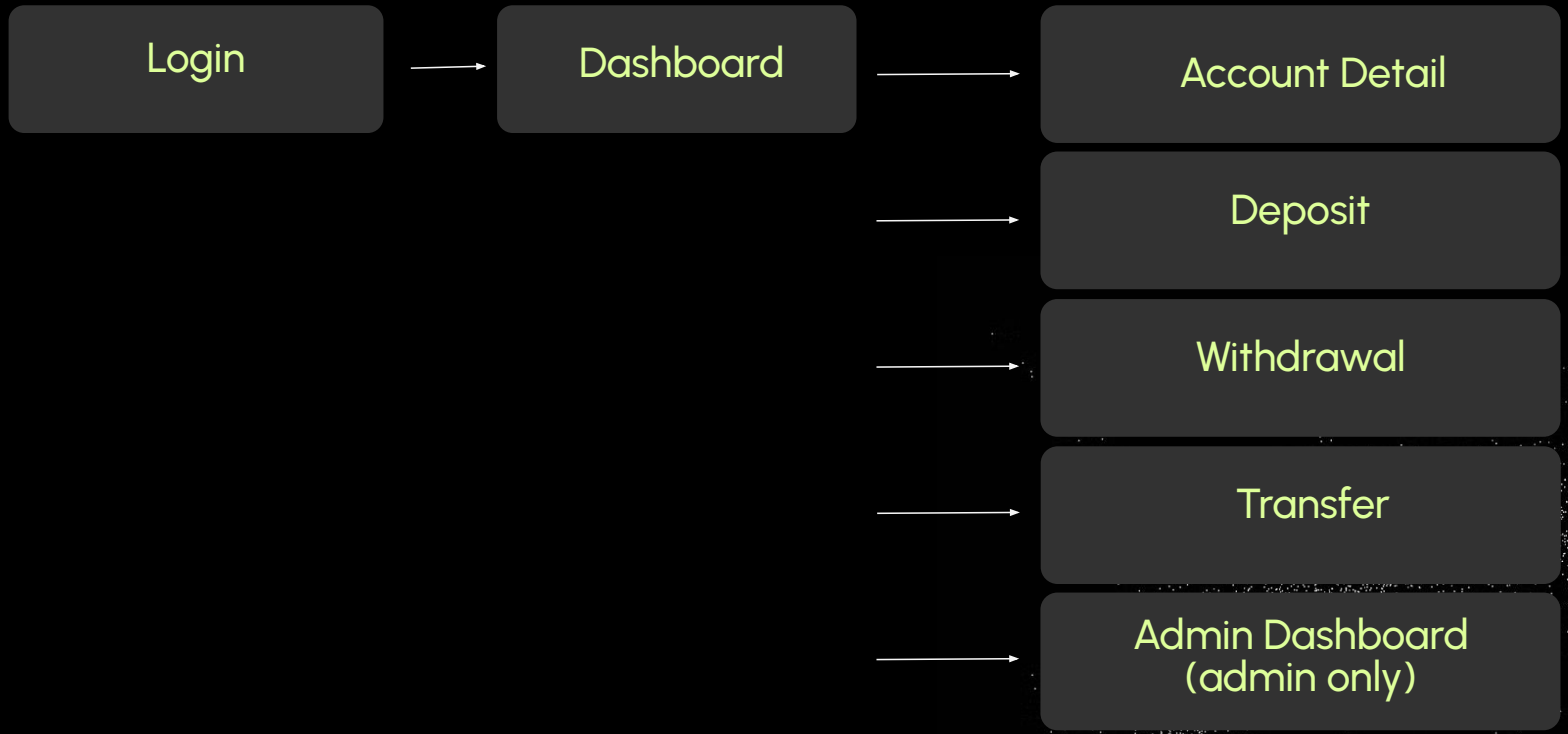
Relationships:

- One user -> multiple accounts
- One account -> multiple transactions

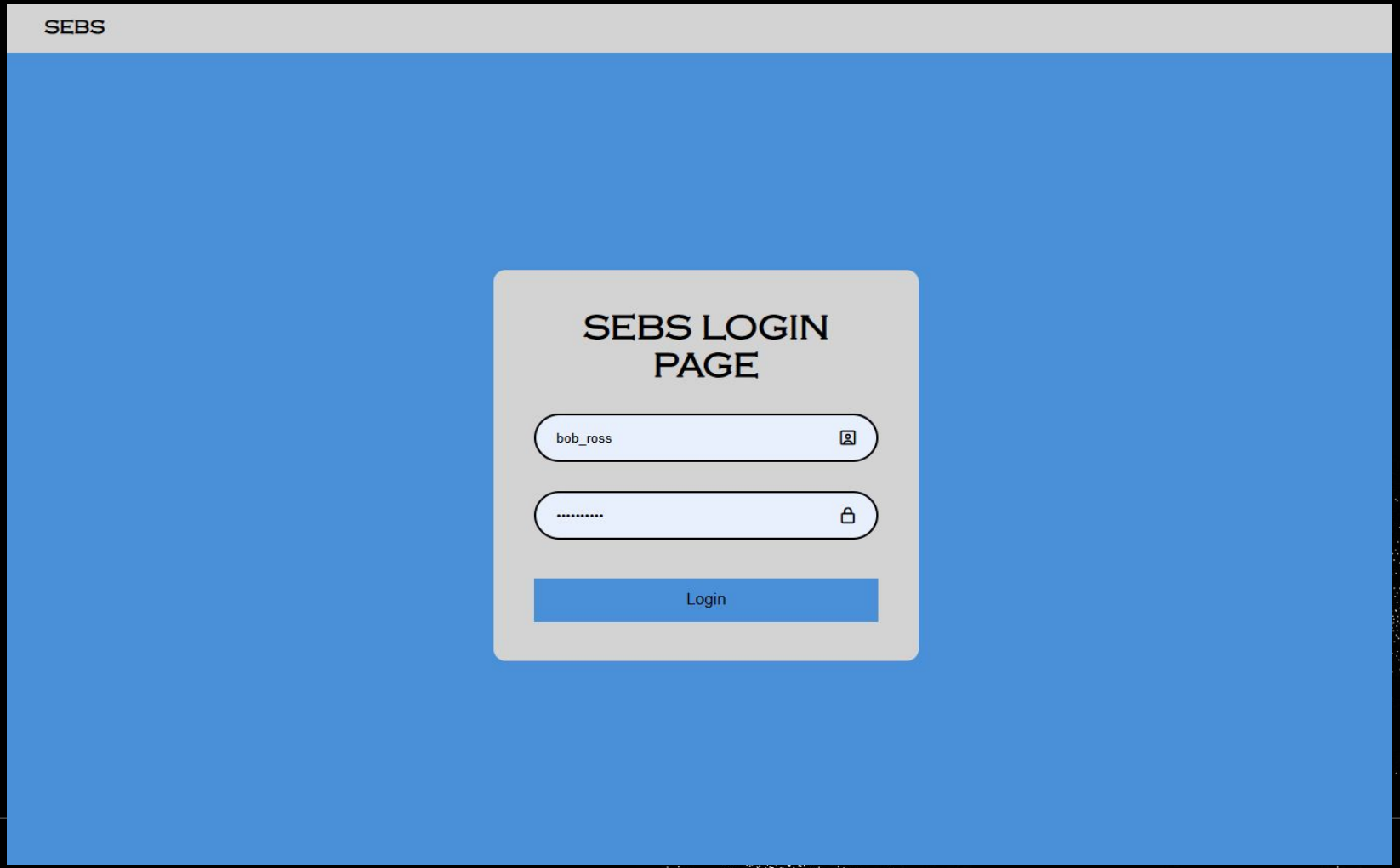
System Workflow (Example)

1. User enters login credentials
2. System authenticates the user
3. User is redirected to their dashboard
4. User selects an account action
5. System processes transaction
6. Updated balance is displayed after transaction completed

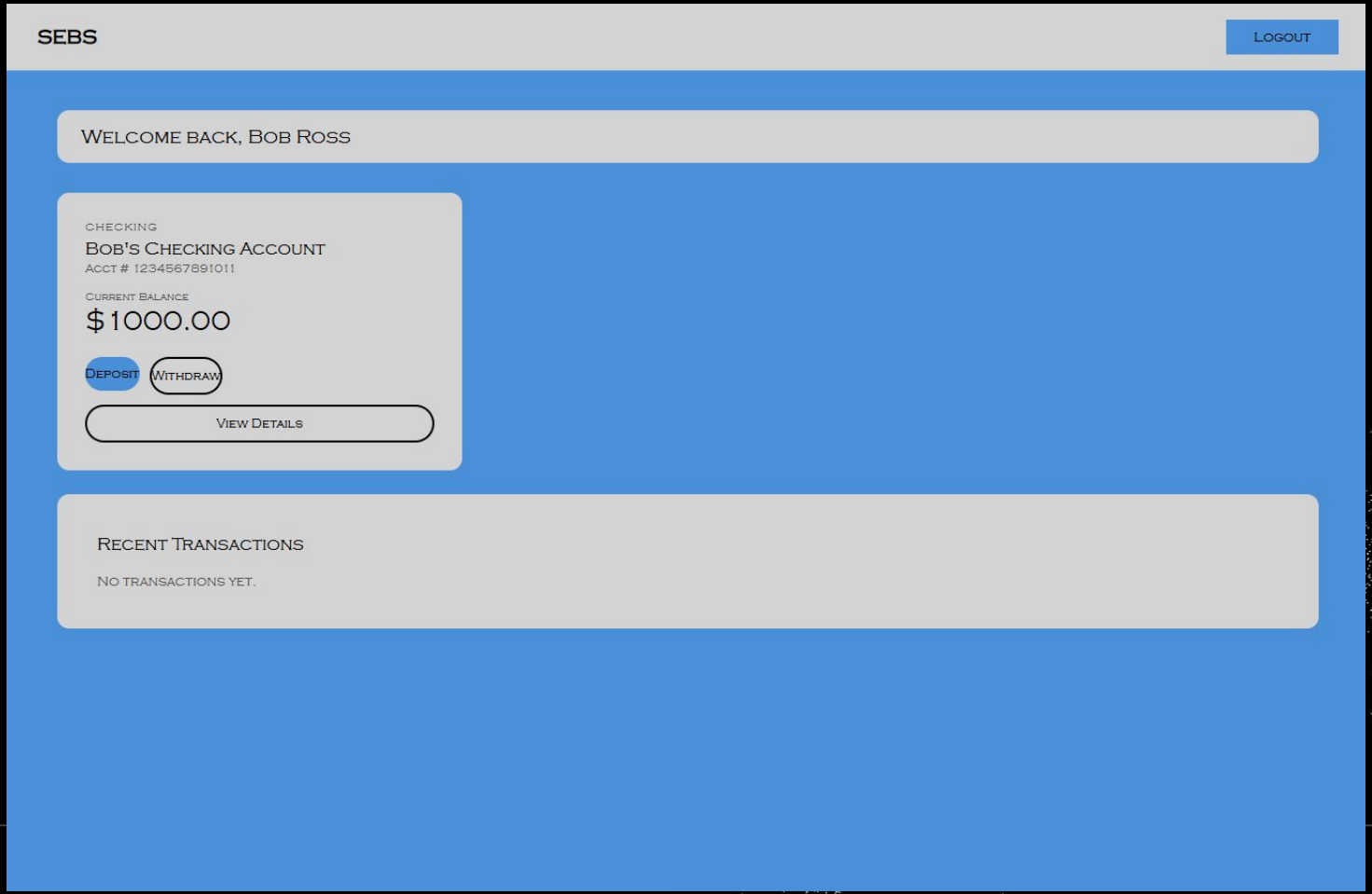
UI Design Overview



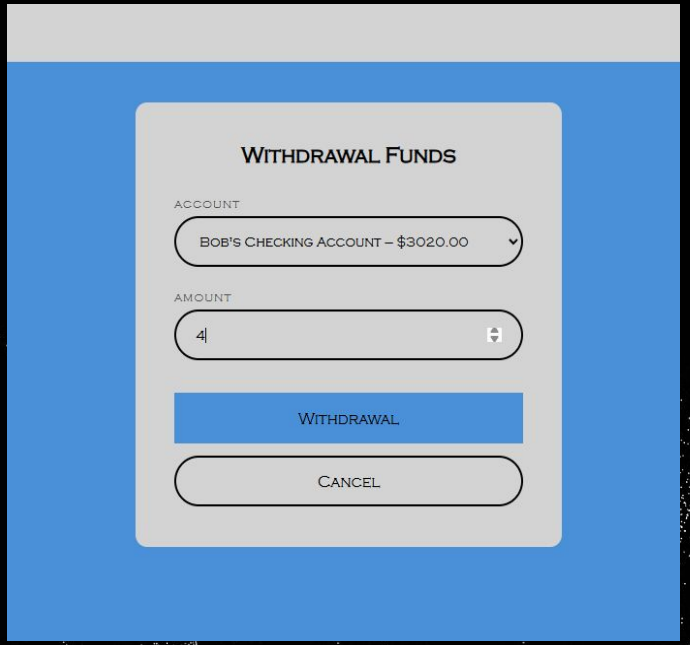
Login Page



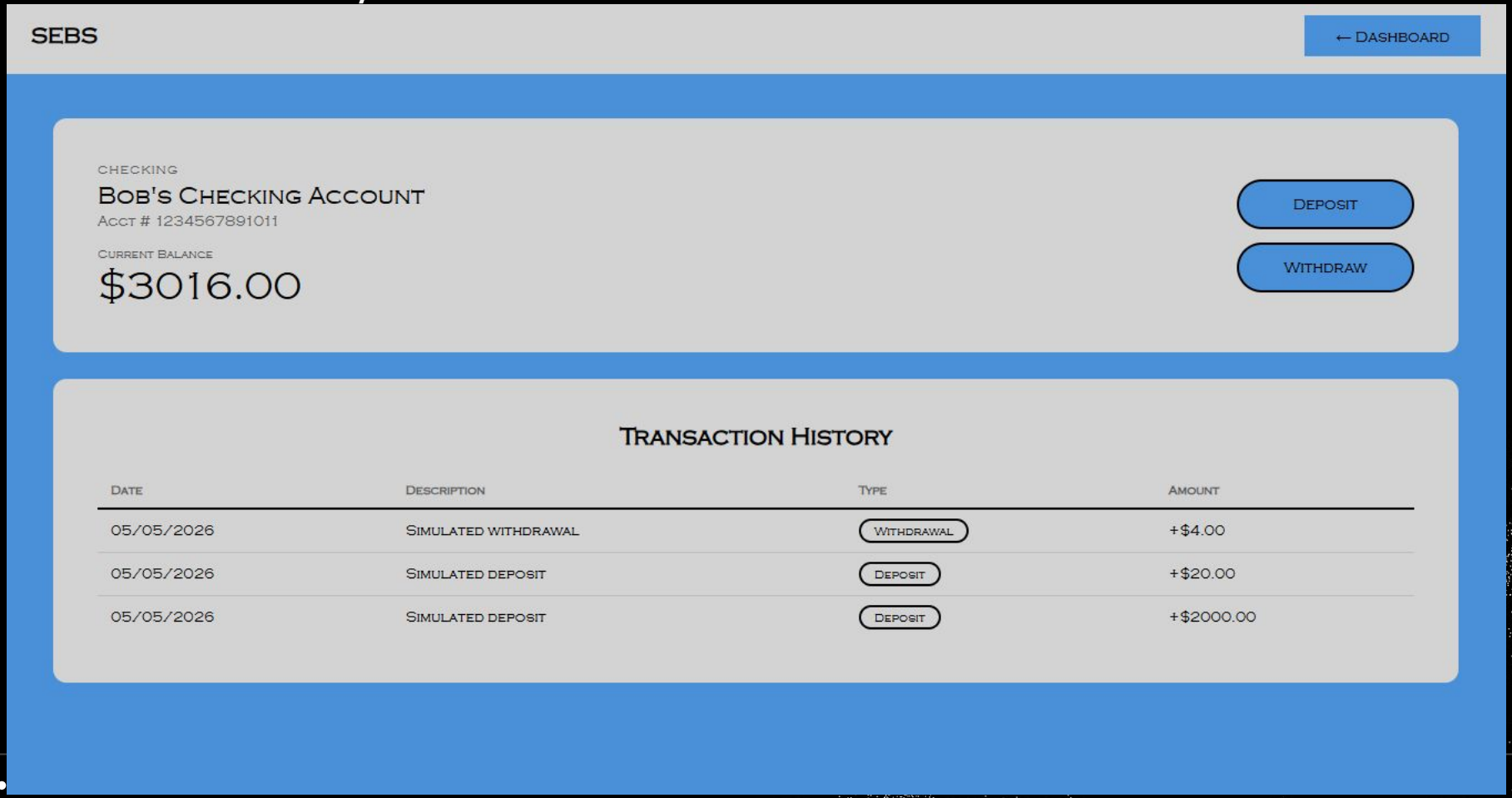
Account Dashboard



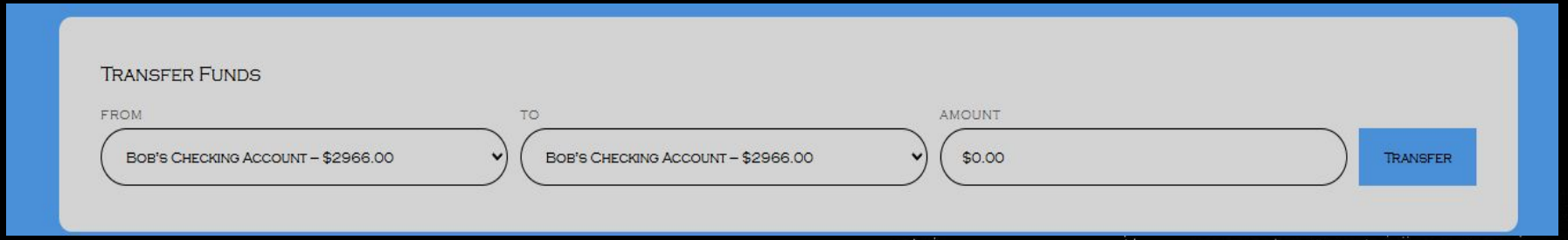
Deposit and Withdrawal



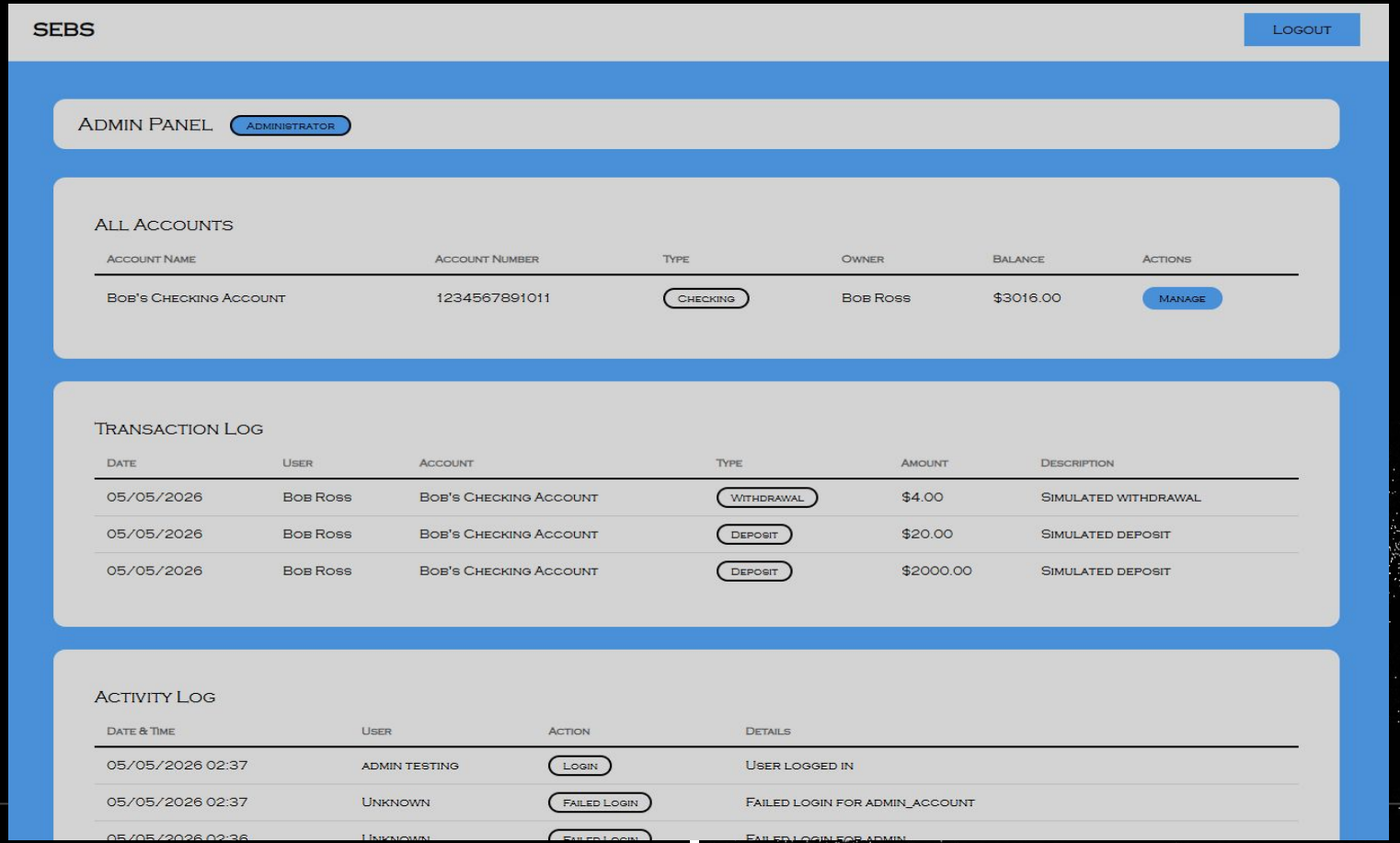
Transaction History



Transfer Funds



Admin Dashboard



Project Status

Completed

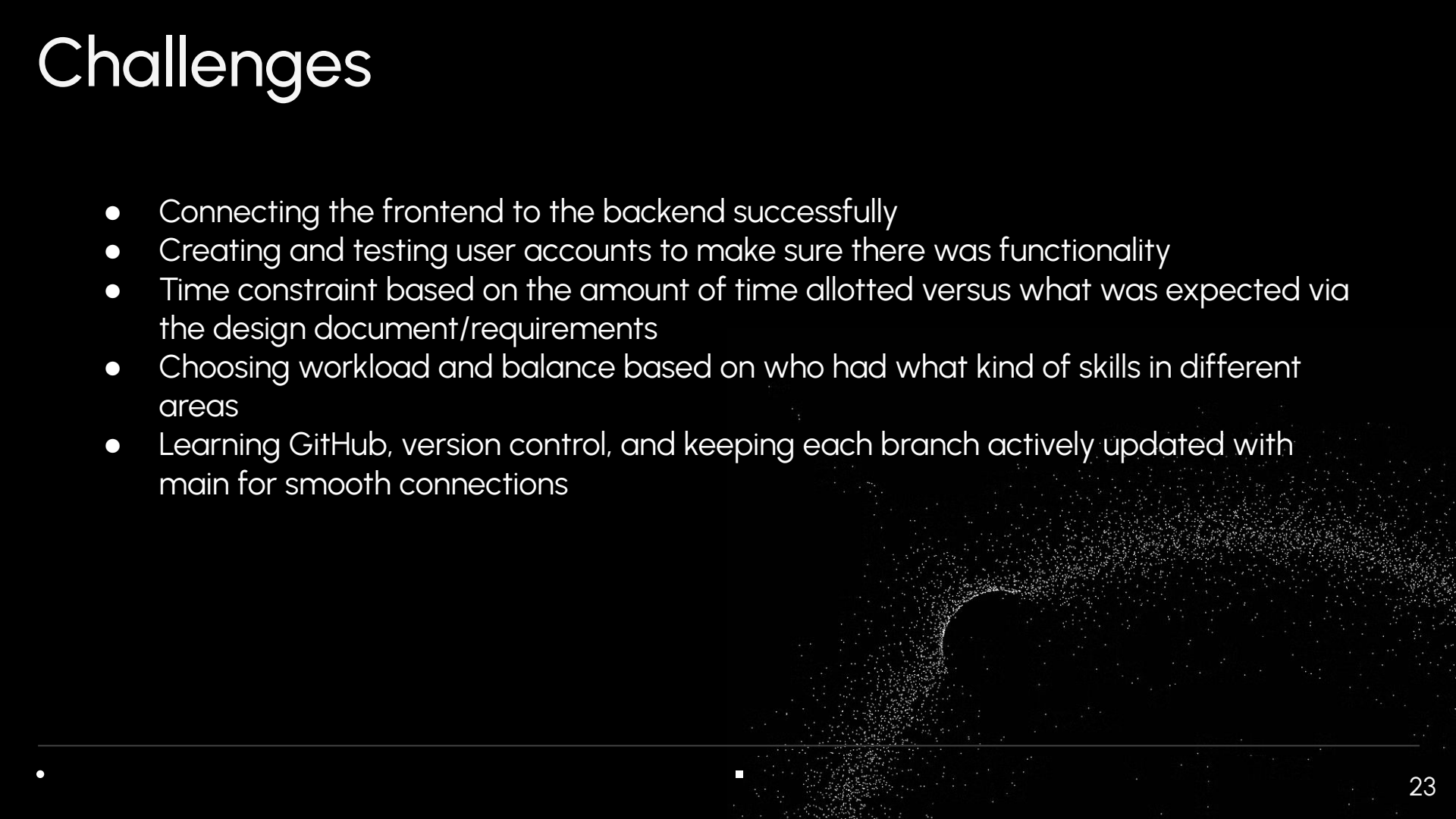
- All four database models (UserProfile, Account, Transaction, ActivityLog)
- Full services layer with deposit, withdraw, and transfer logic
- All view functions working
- All HTML templates exist and render
- Login and logout with activity logging
- Transfer between accounts
- Dashboard showing accounts and balances

Not Yet Completed

- Admin UI for creating and modifying customer accounts through SEBS instead of Django's admin panel
- Requirements.txt needs to be revamped to work properly for any user trying the website for themselves

Challenges

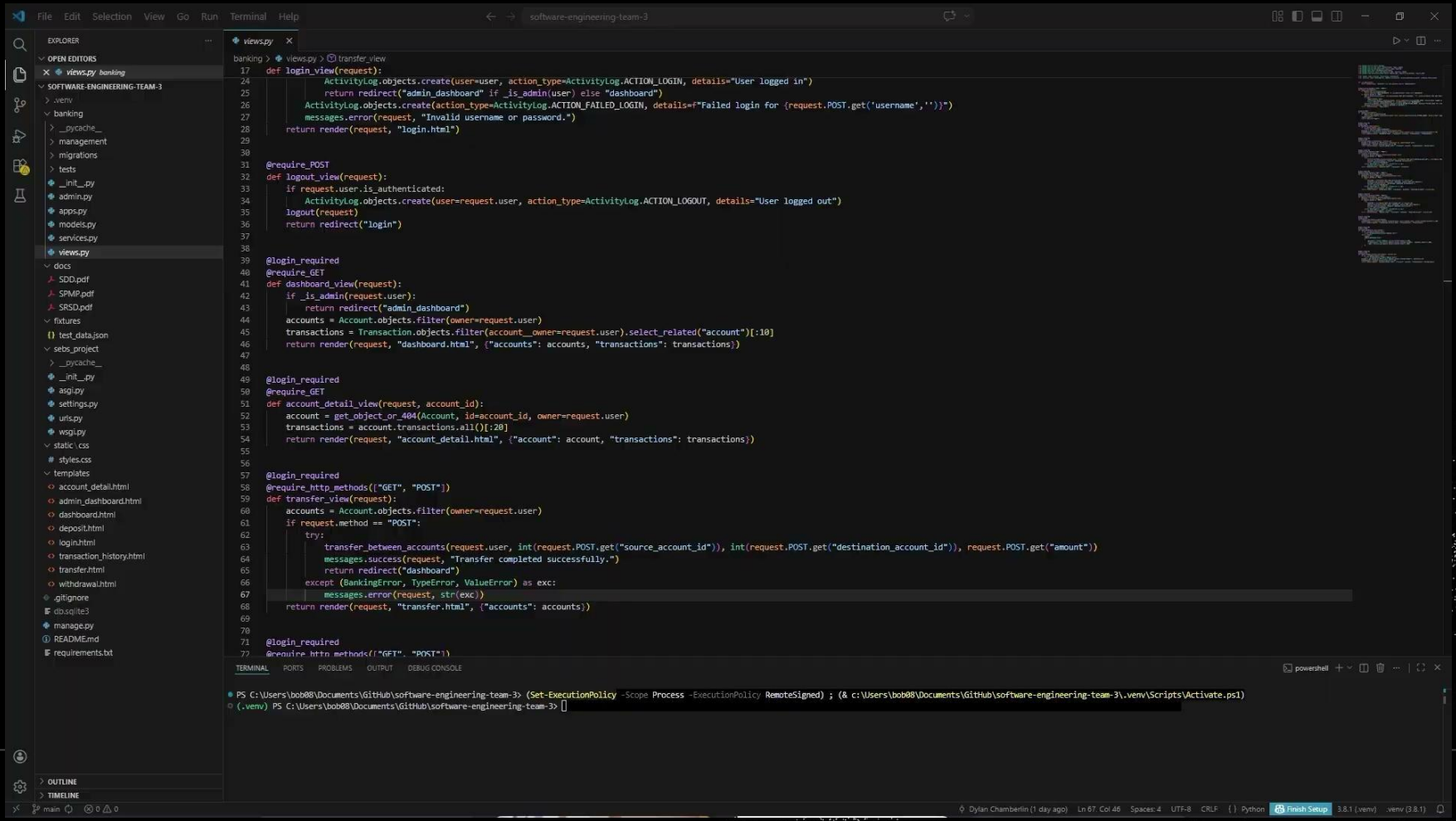
- Connecting the frontend to the backend successfully
- Creating and testing user accounts to make sure there was functionality
- Time constraint based on the amount of time allotted versus what was expected via the design document/requirements
- Choosing workload and balance based on who had what kind of skills in different areas
- Learning GitHub, version control, and keeping each branch actively updated with main for smooth connections



Lessons Learned

- Planning and communication
  - Everyone on the team being able to understand the full stack, learning how the backend connected to the front end
- The development process
  - Testing the app end-to-end as we went instead of just doing it right at the end
- Committing the working code to main regularly, making sure branches weren't out of sync during testing
- Learning Django and it's admin panel
  - Creating test users, accounts, and transactions to simulate a real banking experience

Demo


File Edit Selection View Go Run Terminal Help
software-engineering-team-3
EXPLORER
OPEN EDITORS
views.py banking
SOFTWARE-ENGINEERING-TEAM-3
.venv
banking
__pycache__
management
migrations
tests
__init__.py
admin.py
apps.py
models.py
services.py
views.py
docs
SDD.pdf
SPMP.pdf
SRSD.pdf
fixtures
test_data.json
sebs_project
__pycache__
__init__.py
asgi.py
settings.py
urls.py
wsgi.py
static\css
styles.css
templates
account_detail.html
admin_dashboard.html
dashboard.html
deposit.html
login.html
transaction_history.html
transfer.html
withdrawal.html
.gitignore
db.sqlite3
manage.py
README.md
requirements.txt
views.py
banking > views.py > transfer_view
def login_view(request):
ActivityLog.objects.create(user=user, action_type=ActivityLog.ACTION_LOGIN, details="User logged in")
return redirect("admin_dashboard" if _is_admin(user) else "dashboard")
ActivityLog.objects.create(action_type=ActivityLog.ACTION_FAILED_LOGIN, details=f"Failed login for {request.POST.get('username','')}")
messages.error(request, "Invalid username or password.")
return render(request, "login.html")
@require_POST
def logout_view(request):
if request.user.is_authenticated:
ActivityLog.objects.create(user=request.user, action_type=ActivityLog.ACTION_LOGOUT, details="User logged out")
logout(request)
return redirect("login")
@login_required
@require_GET
def dashboard_view(request):
if _is_admin(request.user):
return redirect("admin_dashboard")
accounts = Account.objects.filter(owner=request.user)
transactions = Transaction.objects.filter(account__owner=request.user).select_related("account")[:10]
return render(request, "dashboard.html", {"accounts": accounts, "transactions": transactions})
@login_required
@require_GET
def account_detail_view(request, account_id):
account = get_object_or_404(Account, id=account_id, owner=request.user)
transactions = account.transactions.all()[:20]
return render(request, "account_detail.html", {"account": account, "transactions": transactions})
@login_required
@require_http_methods(["GET", "POST"])
def transfer_view(request):
accounts = Account.objects.filter(owner=request.user)
if request.method == "POST":
try:
transfer_between_accounts(request.user, int(request.POST.get("source_account_id")), int(request.POST.get("destination_account_id")), request.POST.get("amount"))
messages.success(request, "Transfer completed successfully.")
return redirect("dashboard")
except (BankingError, TypeError, ValueError) as exc:
messages.error(request, str(exc))
return render(request, "transfer.html", {"accounts": accounts})
@login_required
@require_http_methods(["GET", "POST"])
TERMINAL PORTS PROBLEMS OUTPUT DEBUG CONSOLE
powershell
PS C:\Users\bob08\Documents\GitHub\software-engineering-team-3> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\bob08\Documents\GitHub\software-engineering-team-3\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\bob08\Documents\GitHub\software-engineering-team-3>
OUTLINE
TIMELINE
main
Dylan Chamberlin (1 day ago) Ln 67 Col 46 Spaces: 4 UTF-8 CRLF { } Python Finish Setup 3.8.1 (.venv) .venv (3.8.1)

Thanks!